

ngspice-46 vs. Spectre — Feature Gap Analysis

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice-46 vs. a commercial simulator (Spectre) — feature gap analysis

1

ngspice-46 vs. a commercial simulator (Spectre) — feature gap analysis

A capability comparison of the ngspice-46 build in this repository against a Spectre-class commercial simulator, to show where the open-source tool already matches the commercial one and where the gaps are. Grounded in a direct read of the ngspice-46/src/ tree (analyses, solver, devices, RF, convergence, parallelism), not marketing feature lists.

Legend: ✓ present / on par · ⚠ partial, experimental, or via a workaround · ✗ absent. "Spectre" stands in for the commercial reference (Spectre / SpectreRF / Spectre X / RelXpert feature set).

The one column that matters is ngspice — Spectre has essentially everything, so its column is a baseline of ✓. The point of the table is where ngspice's mark is ⚠ or ✗.

Context: the Verilog-A / OSDI device side is not a gap — thanks to the OpenVAF-reloaded compiler in this repository it is on par with (often ahead of) commercial Verilog-A support. The gaps below are all on the simulator/analysis side.

Core numerics

Category	Feature	ngspice	Spectre
Core solver	Sparse LU (KLU) direct solver	✓	✓
Core solver	Legacy Sparse1.3 fallback	✓	✓
Core solver	Parallel / partitioned / GPU linear solve	✗	✓
Core solver	Matrix reordering + scaling beyond KLU defaults	⚠	✓
Integration	Trapezoidal + variable-order Gear	✓	✓
Integration	Advanced LTE-based step/order control	⚠	✓
Convergence	gmin stepping + source stepping homotopy	✓	✓
Convergence	Pseudo-transient / dynamic-gmin continuation	✗	✓
Convergence	Damped / trust-region (globalized) Newton	✗	✓
Convergence	Coordinated accuracy presets (errpreset)	✓	✓

Standard analyses (analog)

Category	Feature	ngspice	Spectre
Analyses	DC operating point / DC sweep	✓	✓
Analyses	Transient (.tran)	✓	✓
Analyses	AC small-signal (.ac)	✓	✓
Analyses	Noise, small-signal (.noise, onoise/inoise + integrated)	✓	✓
Analyses	Pole-zero (.pz)	✓	✓
Analyses	Transfer function (.tf)	✓	✓
Analyses	DC sensitivity (.sens)	✓	✓

Category	Feature	ngspice	Spectre
Analyses	Distortion (.disto)	✓	✓
Analyses	Measurement / post-processing (.meas)	✓	✓

RF / periodic steady-state suite

Category	Feature	ngspice	Spectre
RF	S-parameter analysis + noise figure (.sp)	✓	✓
RF	Touchstone (.sNp) import / export	✓	✓
RF	Periodic steady state (PSS)	⚠	✓
RF	Harmonic Balance (HB)	×	✓
RF	Periodic / phase noise (Pnoise)	×	✓
RF	Periodic AC (PAC, conversion gain)	×	✓
RF	Periodic transfer function (PXF)	×	✓
RF	Periodic S-parameters (PSP)	×	✓
RF	Quasi-periodic / multi-tone (QPSS / QPAC)	×	✓
RF	Envelope following	×	✓

PSS is ⚠ present but a brute-force shooting method, flagged experimental, and fragile on some circuits. HB exists only as a WITH_HB stub that returns "unsupported".

Performance & scale

Category	Feature	ngspice	Spectre
Performance	Multithreaded device-model evaluation (OpenMP)	✓	✓
Performance	Element bypass / latency exploitation	⚠	✓
Performance	Fast-SPICE hierarchical / isomorphic engine	×	✓
Performance	Distributed (MPI) / cloud partitioning	×	✓
Performance	Handles 10 ⁶ –10 ⁸ -node post-layout netlists	×	✓

Statistical / variability / yield

Category	Feature	ngspice	Spectre
Statistical	Monte Carlo	⚠	✓
Statistical	Native process/mismatch modeling + correlations	⚠	✓
Statistical	Low-discrepancy sampling (Sobol / Latin-hypercube)	×	✓
Statistical	High-sigma methods (importance sampling, worst-case distance)	×	✓
Statistical	Corner + MC + yield estimation flow	⚠	✓

Monte Carlo is ⚠ works, but script-driven (alter + sgauss, with the deterministic-seed RNG from Enhancement-10/66) rather than a native statistical block with sampling controls.

Reliability / aging

Category	Feature	ngspice	Spectre
Reliability	Device aging (HCI / NBTI / TDDDB)	×	✓
Reliability	Stress → degrade → re-simulate (fresh/aged) flow	×	✓
Reliability	Electromigration + IR-drop (EMIR)	×	✓

Post-layout / parasitics

Category	Feature	ngspice	Spectre
Post-layout	Flat parasitic (RC) netlist simulation	✓	✓
Post-layout	RC reduction / model-order reduction	✗	✓
Post-layout	n-port (S/Y/Z) extracted-block import	✓	✓

Behavioral / mixed-signal / AMS

Category	Feature	ngspice	Spectre
Modeling	Verilog-A (analog, LRM Annex C) via OSDI	✓	✓
Modeling	XSPICE code models + event-driven digital	✓	✓
Modeling	Verilog-AMS mixed-signal (connect modules)	✗	✓
Modeling	Real-number modeling (wreal / RNM)	✗	✓
Modeling	VHDL-AMS	✗	✓

Interfaces & infrastructure

Category	Feature	ngspice	Spectre
Interface	Shared-library / programmatic API	✓	✓
Interface	Python bindings	✓	✓
Infrastructure	Built-in optimizer	✗	✓
Infrastructure	Checkpoint / restart of long runs	⚠	✓

Where to invest (given the Verilog-A/OSDI side is done)

The realistic, winnable identity for this project is the open-source, OSDI/Verilog-A-native simulator with a real RF-noise and statistical story — not out-engineering 25 years of commercial fast-SPICE/parallel work. Ranked by leverage on the existing strength $\times$ differentiation $\times$ tractability:

1. RF periodic small-signal suite — Pnoise $\rightarrow$ PAC $\rightarrow$ PXF, on a hardened PSS. The device/OSDI side already supplies what these need (noise-source topology, operating-point- and frequency-dependent load_noise, periodic Jacobians); the work is the analysis engines. Genuinely novel in open source.
2. Convergence robustness — coordinated accuracy presets (errpreset) landed in [Enhancement-110](#); the remaining piece is pseudo-transient / dynamic-gmin homotopy. Unglamorous but makes every analysis usable on real circuits; self-contained in the ngspice core.
3. High-sigma statistical sampling — high industrial value (yield / SRAM), moderate difficulty, leans on the deterministic-seed RNG already in place.

Explicitly lower priority: fast-SPICE parallelism and aging/EMIR — enormous efforts that don't leverage what makes this project distinctive.